Retrieval Augmented Language Models (Continued)

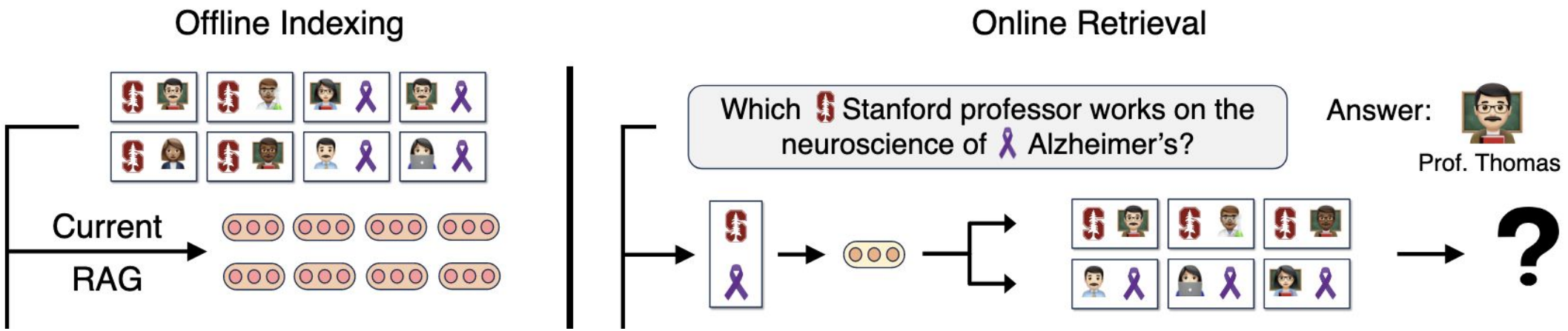
CS728

Soumen Chakrabarti

Yatin Nandwani

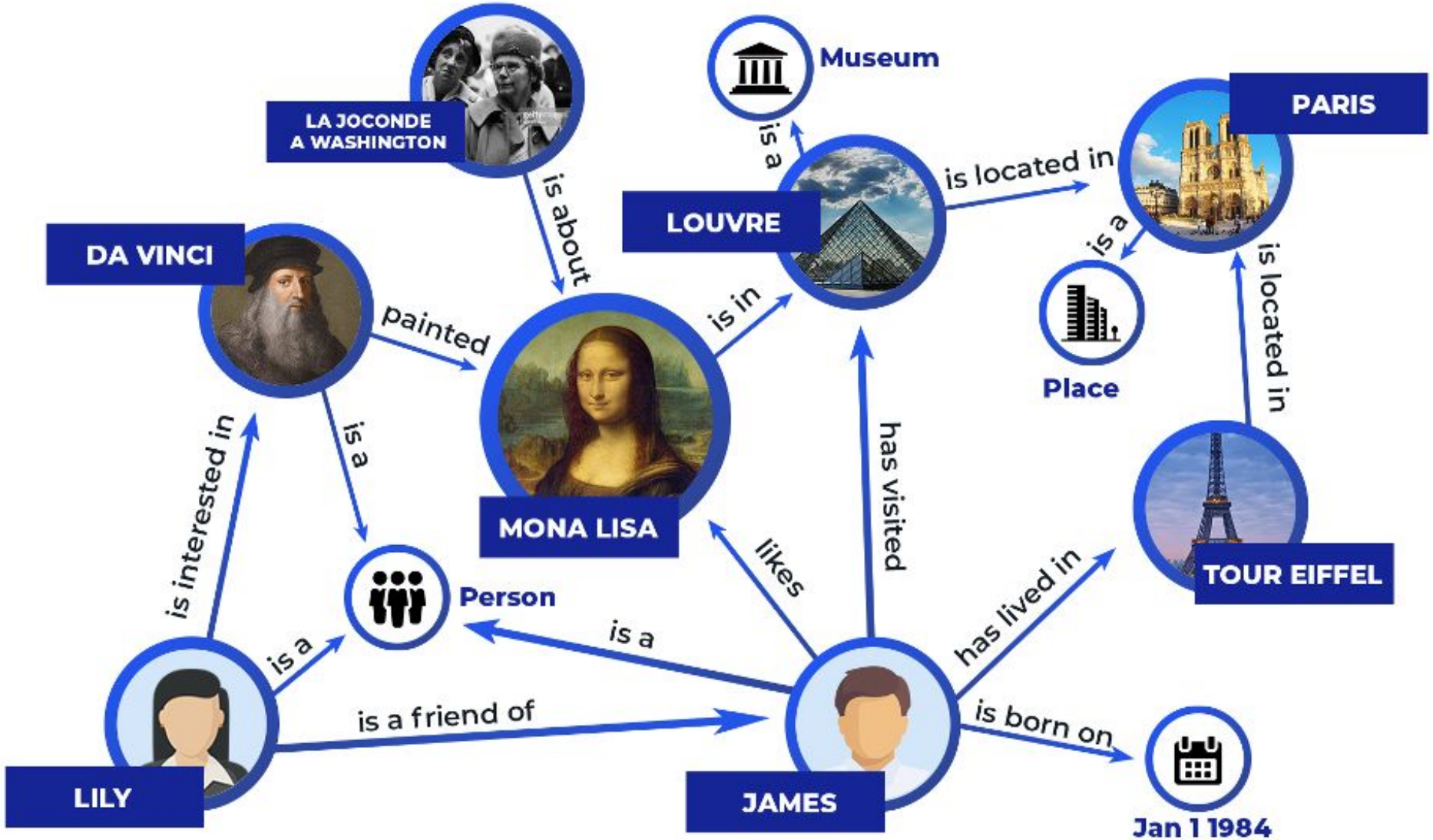
Issues with RAG 1.0

- May fail for complex multi-hop queries

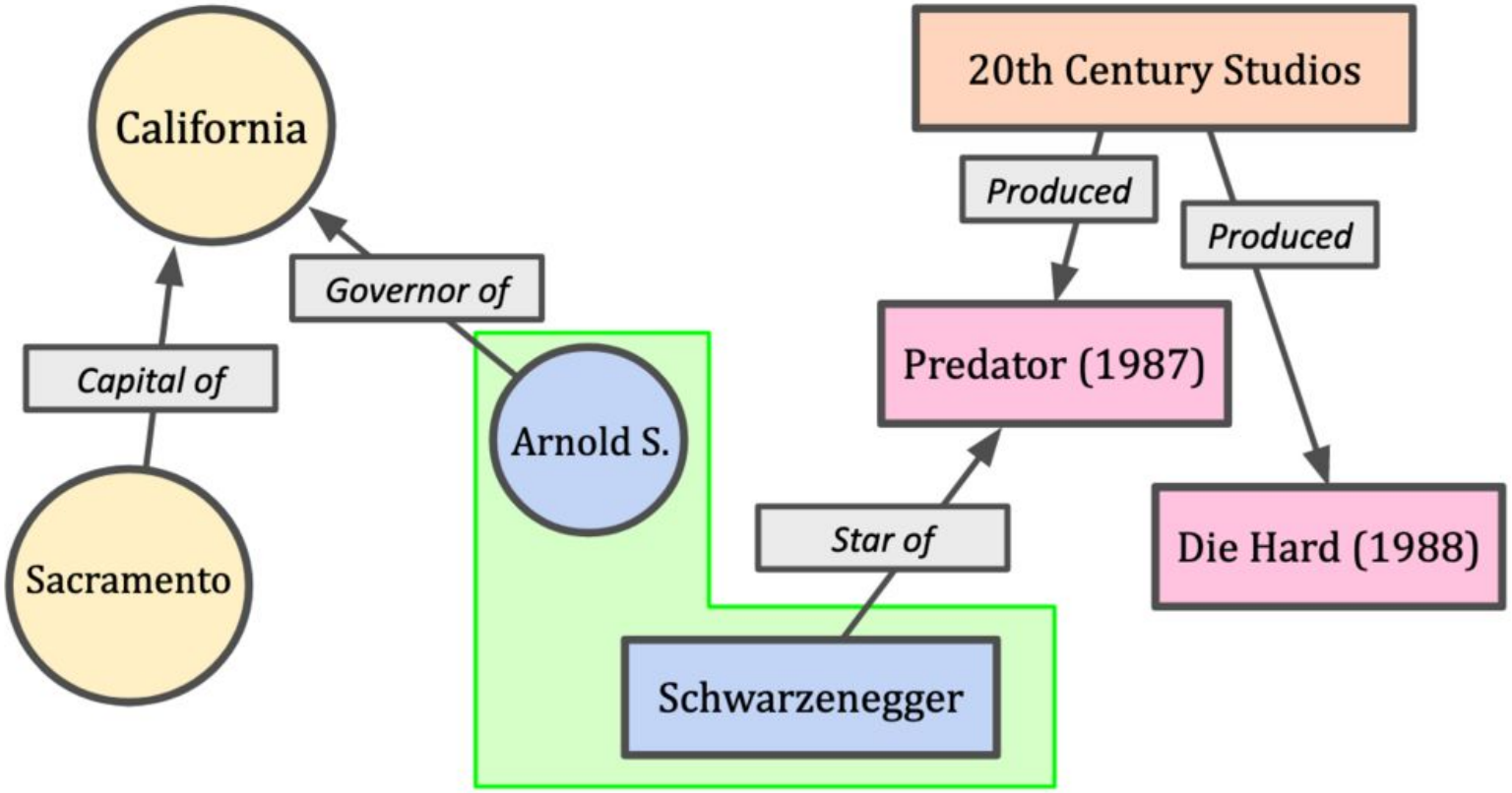


source: HippoRAG: Neurobiologically Inspired Long-Term Memory for Large Language Models

What is a knowledge graph (KG)?



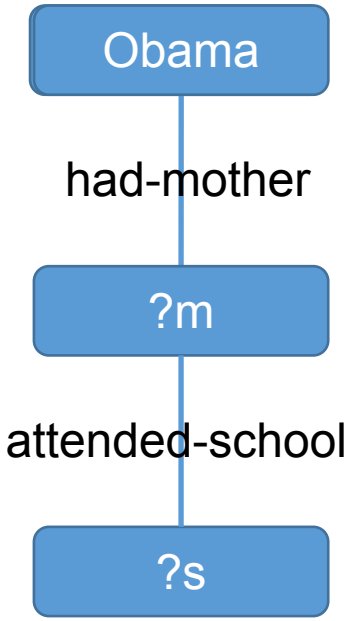
Another KG fragment



How about retrieving from knowledge graphs?

- Which high school did Obama's mother attend?
- KGs are a set of (subject, relation, object) triples
- Fields are opaque IDs, not strings
  - Mapping from ID to strings in multiple languages
- Question decomposition and multi-hop reasoning needed
  - Question can be represented as an incomplete graph
- KG has hundred billion edges
  - Too big for LLM context, must retrieve; how?

Subject	Relation	Object
Obama	attended-school	Punahou
Obama	had-mother	Ann Dunham
Ann Dunham	attended-school	Eckstein Junior High



A generic pseudocode using “retrieval tool”

- Preprocess and index corpus; encode prompt
- Go into “thinking tokens” mode
- For every “thinking token”
  - Decide if LLM needs to retrieve
  - If yes, compose a query for the “retrieval tool”
  - Paste returned docs/passages into context
- Switch to “reasoning and generation” mode
  - Emit response
- (May move between thinking, retrieval, reasoning states multiple times before final generation)

Linking, retrieval, reasoning, generation

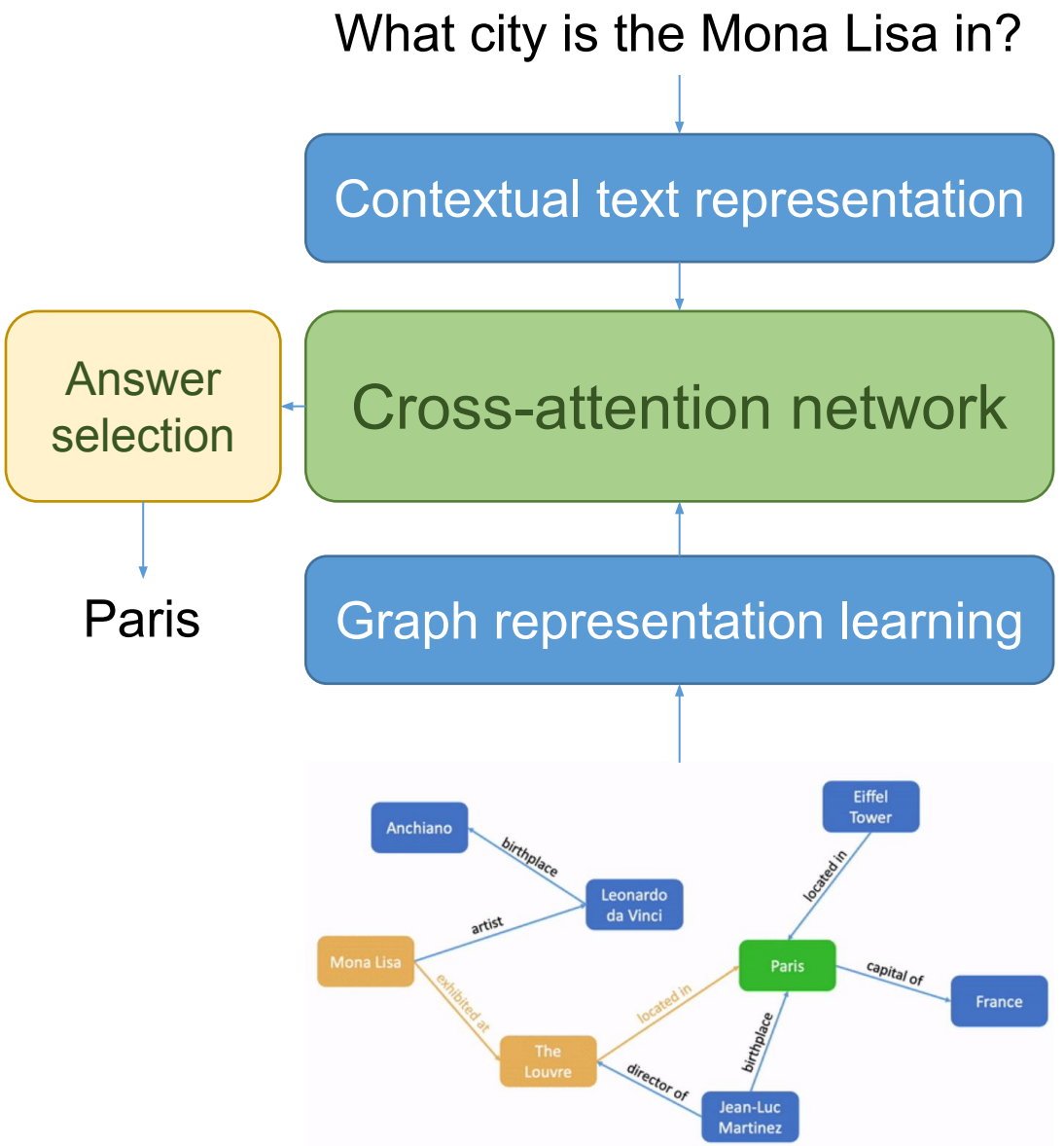
- Need to link question to entities in KG
  - Disambiguation of John Smith
  - Oblique reference “first president”, “red planet”
  - Usually also need to link relations
- In general, need to retrieve a subgraph of the KG
  - Contains linked entities and answer elements
  - Plus evidence nodes/edges in favor of answers
- Reasoning and generation use retrieved subgraph
  - May be transcribed to text for context injection

Retrieval from KG

- "Differentiable KG"
- Adapter between LM and KG
  - KnowBERT, KAdapter, QA-GNN
- Transcribe KG to text and fine-tune LM
  - KG-BERT, KGLM
- Transcribe KG to text at test time
  - KGT5, KGT5context
- KG completion with/out corpus support
  - IR4KGC, ReSKGC

“Differentiable KG”

- Find dense query embedding (e.g., using BERT)
- Find dense graph embedding (e.g., GCN, graph transformer)
- Cross attention layers establish correspondences between query and graph elements
- Answer selection identifies graph element(s) to transcribe to response
  - End-to-end trainable, no logical form needed
  - Not easily interpretable
  - Text and graph embeddings must play well together



KVMNet/KVMemNN: Explore KG chains

Subject	Relation	Object
Obama	attended-school	Punahou
Obama	had-mother	Ann Dunham
Ann Dunham	attended-school	Eckstein Junior High

Distribution / "attention" over all rows of the KG

Key

Value

q_0

v_1

Attention-weighted value vector, close to Ann Dunham's KG representation

q_1

Query encoder

Query vector for next round

Where did Obama's mother attend school?

KVMNet: Second hop

Subject	Relation	Object
Obama	attended-school	Punahou
Obama	had-mother	Ann Dunham
Ann Dunham	attended-school	Eckstein Junior High

Ke
y

Valu
e

Second hop "attention" over all rows of the KG

q_1

v_2

Answer vector close to "Eckstein Junior High"

Some residue of question "where did ~~Obama's mother~~ attend school" together with Ann Dunham

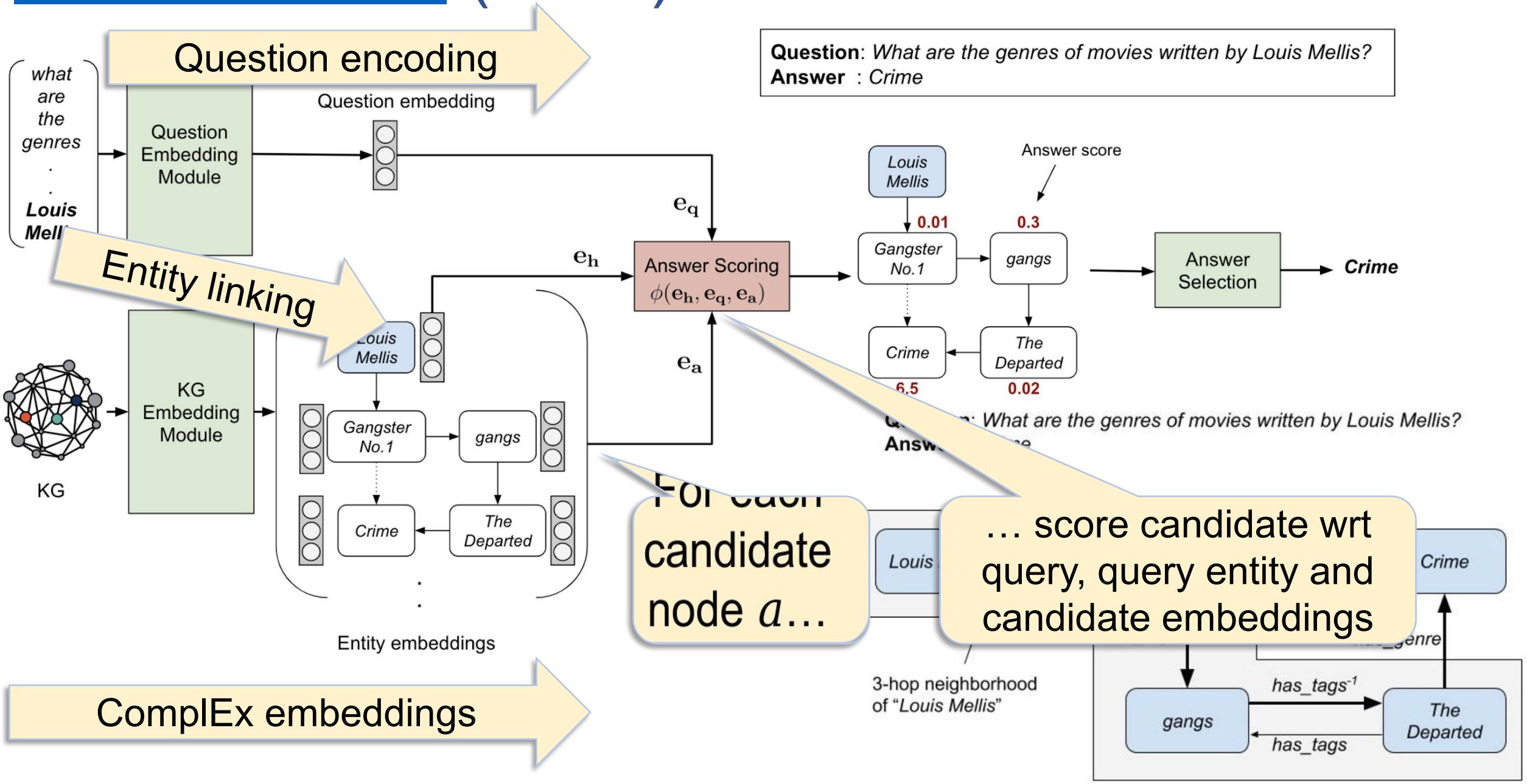


v_1

q_2

"Nothing more to ask"

EmbedKGQA (2020)



Sample results

	1-hop	2-hop	3-hop
KVMNet	96.2	82.7	48.9
GraftNet	97.0	94.8	77.7
PullNet	97.0	99.9	91.4
EmbedKGQA	97.5	98.8	94.8

- MetaQA full-KG benchmark
- PullNet, GraftNet, more sophisticated graph retrieval
- Increasing hop count more difficult for all
- EmbedKGQA (likely) wins from
  - Better knowledge graph embeddings (ComplEx)
  - Better query representation via RoBERTa
  - Fewer moving parts, simpler integration, easier training

Retrieval from KG

- "Differentiable KG"
- Adapter between LM and KG
  - KnowBERT, KAdapter, QA-GNN
- Transcribe KG to text and fine-tune LM
  - KG-BERT, KGLM
- Transcribe KG to text at test time
  - KGT5, KGT5context
- KG completion with/out corpus support
  - IR4KGC, ReSKGC

KnowBERT (2019) — LM adaptation

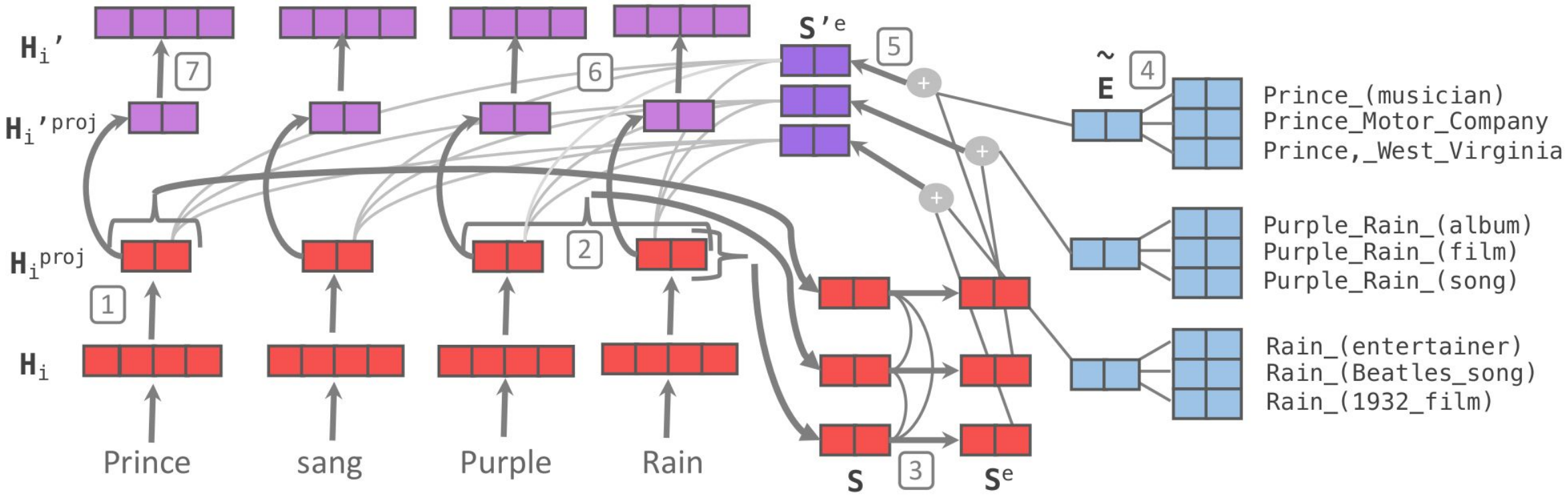


Figure 1: The Knowledge Attention and Recontextualization (KAR) component. BERT word piece representations ($\mathbf{H}_i$) are first projected to $\mathbf{H}_i^{\text{proj}}$ (1), then pooled over candidate mentions spans (2) to compute $\mathbf{S}$, and contextualized into $\mathbf{S}^e$ using mention-span self-attention (3). An integrated entity linker computes weighted average entity embeddings $\tilde{\mathbf{E}}$ (4), which are used to enhance the span representations with knowledge from the KB (5), computing $\mathbf{S}'^e$. Finally, the BERT word piece representations are recontextualized with word-to-entity-span attention (6) and projected back to the BERT dimension (7) resulting in $\mathbf{H}'_i$.

KnowBERT details

- Usually, $\boldsymbol{H}^{(\ell+1)} = \mathrm{MLP}\left(\mathrm{MultiHeadAttn}(\boldsymbol{Q} = \boldsymbol{H}^{(\ell)}, \boldsymbol{K} = \boldsymbol{H}^{(\ell)}, \boldsymbol{V} = \boldsymbol{H}^{(\ell)})\right)$
- $\boldsymbol{H}^{(\ell)}_{\mathrm{proj}} = \boldsymbol{H}^{(\ell)} W_1^{\mathrm{proj}} + \boldsymbol{b}_1^{\mathrm{proj}}$
- Named entity spans like ‘Prince’ and “Purple Rain” identified separately
  - “Self attentive span pooling” (fix one mention span)
  - $\beta_t = \boldsymbol{w}_{\star} \bullet \mathrm{FF}_{\star}\left(\boldsymbol{H}^{(\ell)}_{\mathrm{proj}}[t]\right)$
  - $a_t = \exp(\beta_t) / \sum_{t'} \exp(\beta_{t'})$
  - $\widehat{\boldsymbol{h}} = \sum_{t'} a_{t'} \boldsymbol{h}^{(\ell)}_{t'}$ (t' ranges over span tokens)
  - $\boldsymbol{s} = \left[\boldsymbol{h}^{(\ell)}_{\mathrm{START}}, \widehat{\boldsymbol{h}}, \boldsymbol{h}^{(\ell)}_{\mathrm{END}}\right] \cdots \rightarrow \boldsymbol{S}$, used to ‘link’ entities

There is some attempt to disambiguate to canonical entities

KnowBERT details

- Entity linking: assign probability to each entity
  - Prior (popularity?) probability
  - $\boldsymbol{s} \bullet \boldsymbol{e}$ where $\boldsymbol{e}$ is entity embedding (ComplEx etc.)
  - Entity score = $\text{softmax}_e \text{MLP}(\text{prior}, \ \boldsymbol{s} \bullet \boldsymbol{e})$
- Entity linking supervision can be used here
  - $\widetilde{\boldsymbol{e}}$ = score-weighted average of entity vectors
- Patched span embedding $\boldsymbol{s}' = \boldsymbol{s} + \widetilde{\boldsymbol{e}} \cdots \rightarrow \boldsymbol{S}'$
- Recontextualization (damage control?)
  - $\boldsymbol{H}'^{(\ell)}_{\text{proj}} = MLP\left(\text{MultiHeadAttn}\left(\boldsymbol{H}^{(\ell)}_{\text{proj}}, \boldsymbol{S}', \boldsymbol{S}'\right)\right)$
  - $\boldsymbol{H}'^{(\ell)} = \boldsymbol{H}'^{(\ell)}_{\text{proj}} W_2^{\text{proj}} + \boldsymbol{b}_2^{\text{proj}} + \boldsymbol{H}^{(\ell)}$ (allow skip)

QA-GNN (2021)

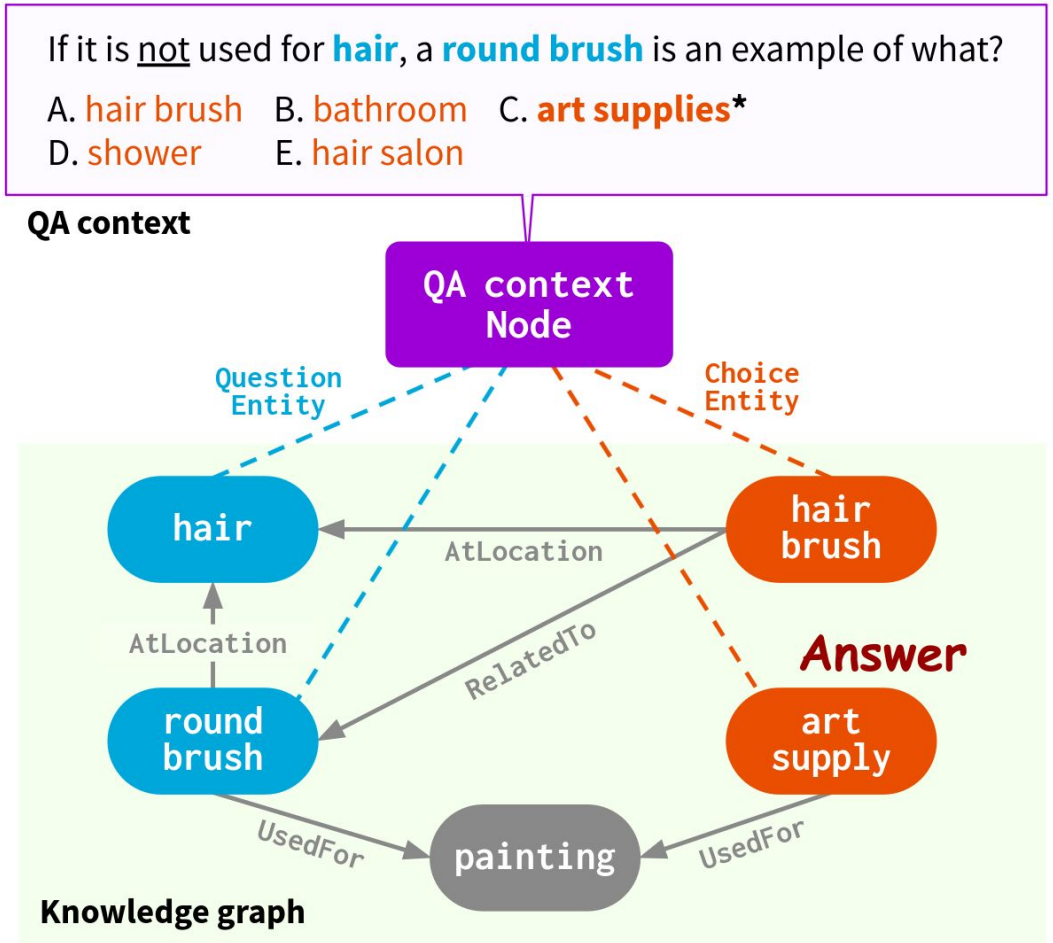
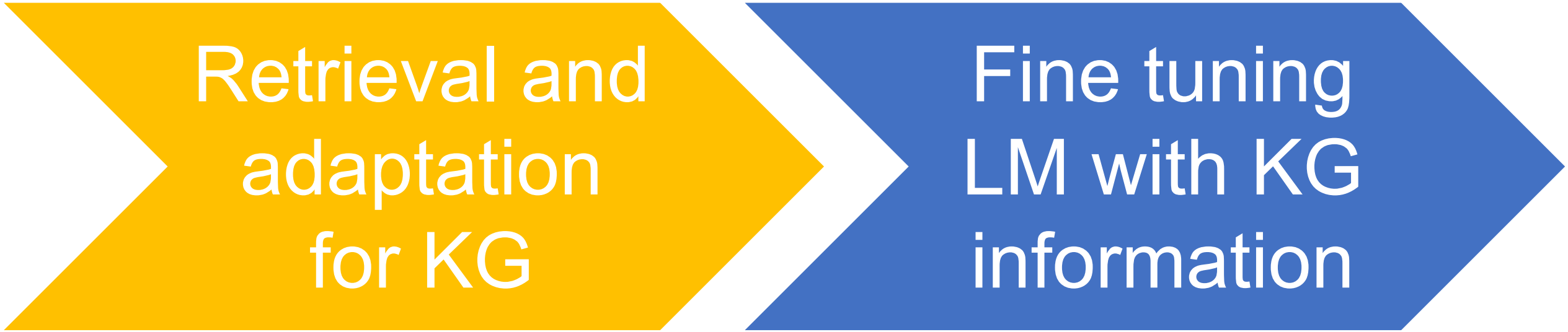


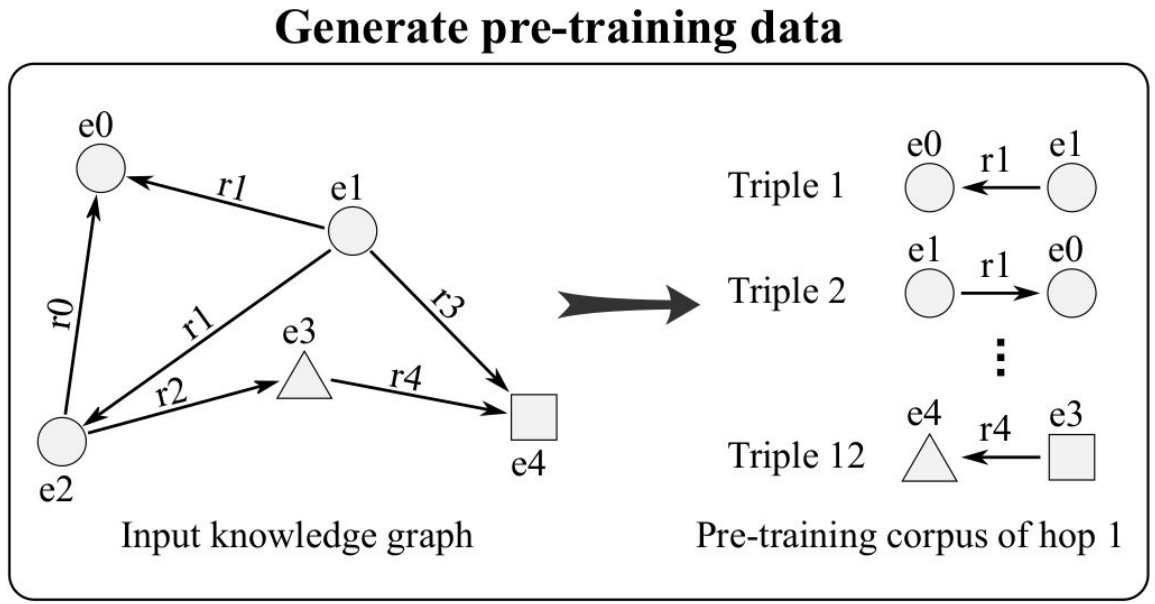
Figure 1: Given the QA context (question and answer choice; purple box), we aim to derive the answer by performing joint reasoning over the language and the knowledge graph (green box).

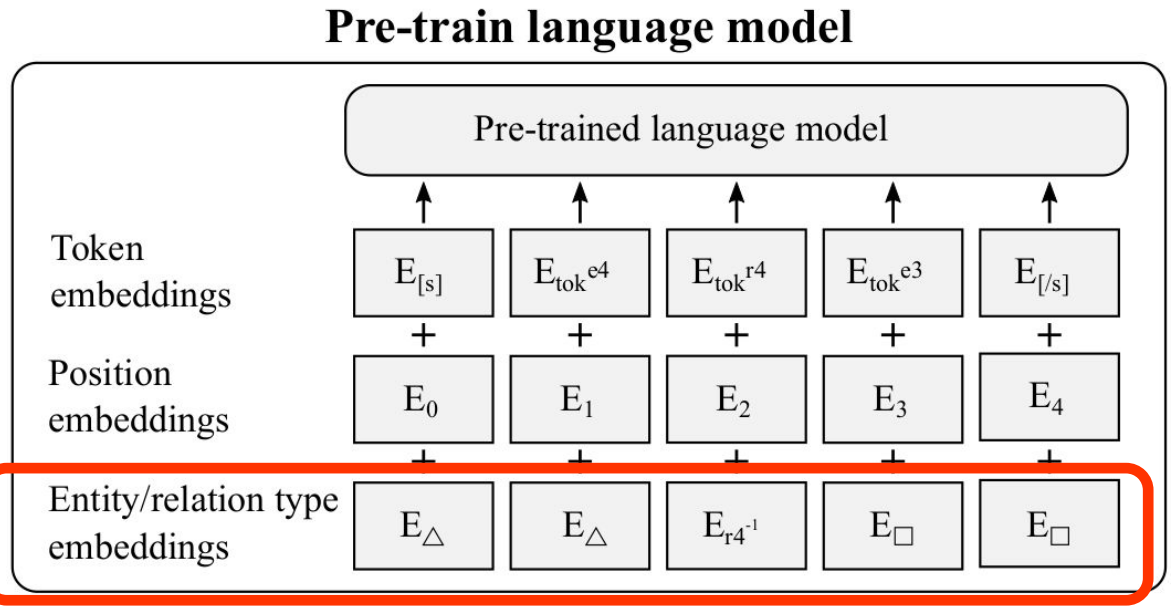
- MCQ style KG-QA
- Question retrieves KG
- Question is also a node
- Each answer node attached in turn
- Resulting graph processed by a GNN
- Combined with question encoding and scored


Retrieval and adaptation for KG
Fine tuning LM with KG information

KGLM (2023)

- Continued pretraining (of RoBERTa)
- Textified KG edges
  - Additional overlay beyond position embeddings
- Sometimes improves on ComplEx for KG completion
  - Not surprising, has more information

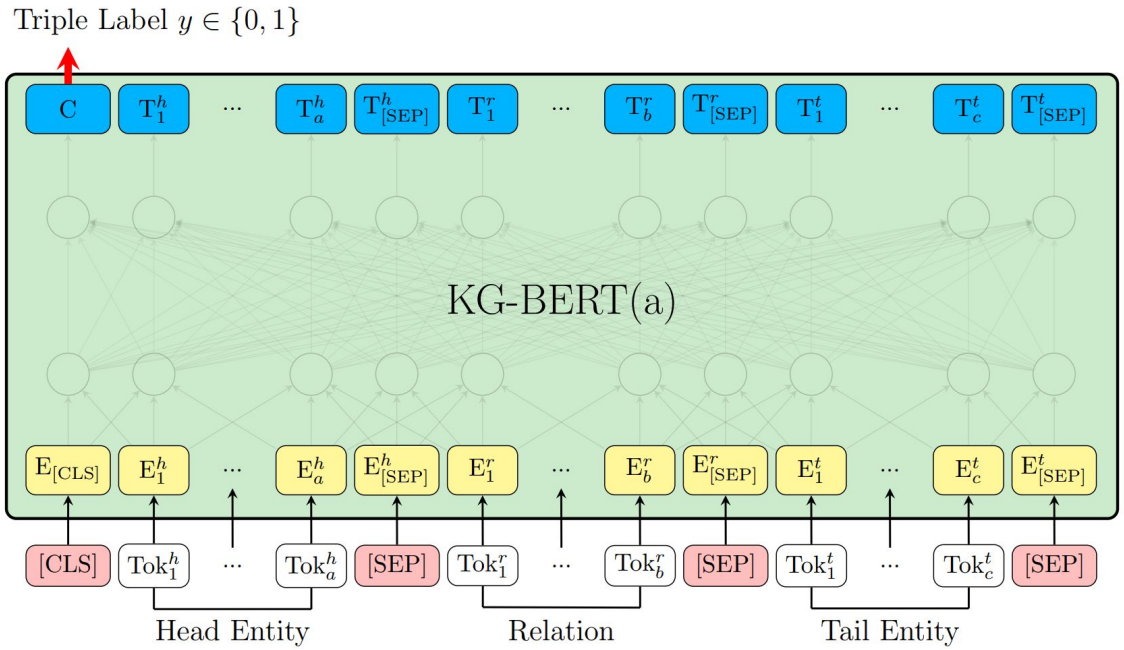


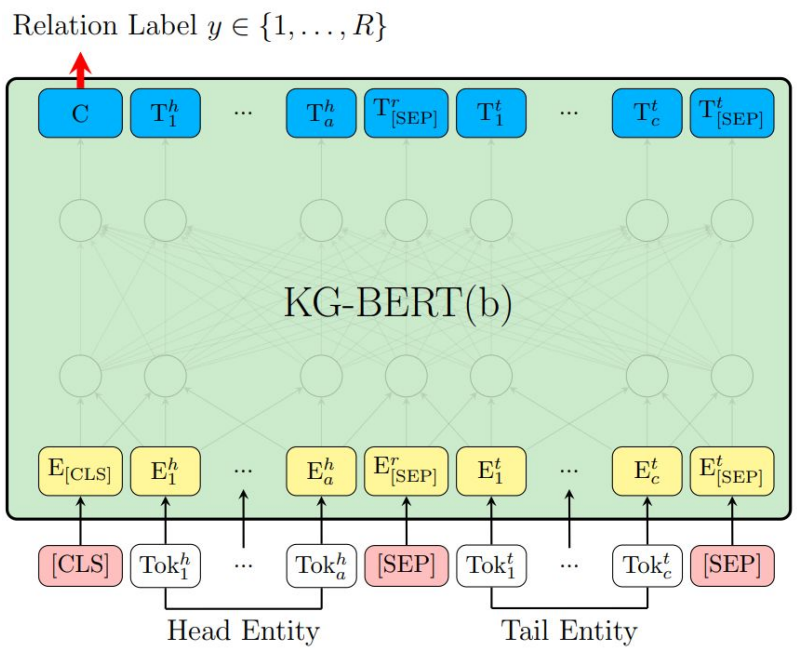


KG-BERT (2019)

- BERT pretrained with masked word and next sentence prediction
- Masking multi-token entities and relations may benefit from some care
- Fine tune with two forms of "KG masking"
  - $\langle s, r, o \rangle \rightarrow \pm 1$ (claim/fact confidence prediction)
  - $\langle s, o \rangle \rightarrow r$ (relation prediction)

KG-BERT (2019)





All inputs are textualized

HITS@10 worse than ComplExN3 SOTA

- Probe BERT with
  - (s, r, o) to get a confidence score
    - Can be used to enumerate and score all candidates o given query $(s, r, ?\, o)$ and vice versa
  - $(s, ?\, r, o)$ to get (canonical) relation label (ordering)
    - Not clear why handled separately
- Test of what relations LM learned by rote from pretraining corpus


Fine tuning LM with KG information
Prompting LM using KG excerpts

KGT5 (2022) and KGT5context (2023)

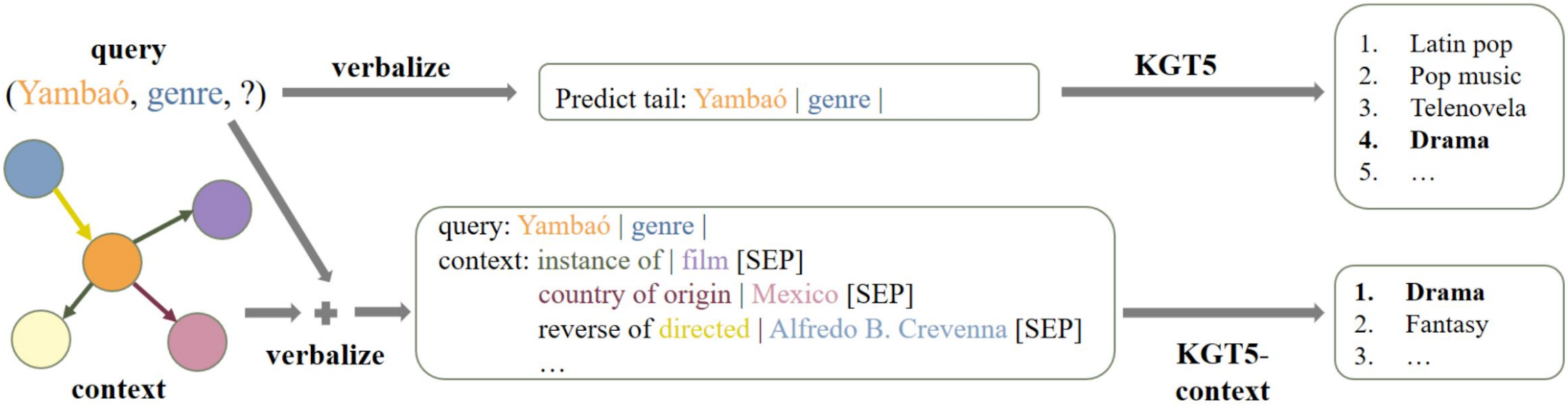
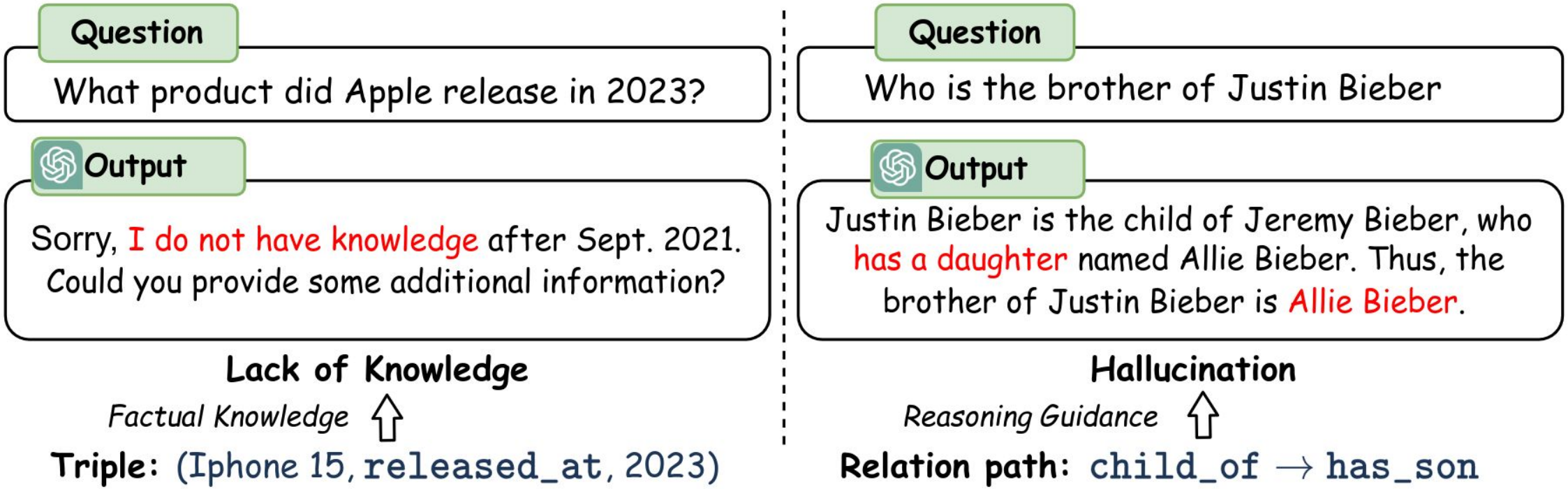


Figure 1: Overview of KGT5-context (at bottom) and comparison to KGT5 (on top); real example from Wikidata5M, best viewed in color. KGT5-context differs from KGT5 in that it appends the neighboring relations and entities of *Yambaó* (a drama movie) to the verbalized query. Both models then apply T5, sample predictions from the decoder, map the samples to entities, and rank by sample logit scores.

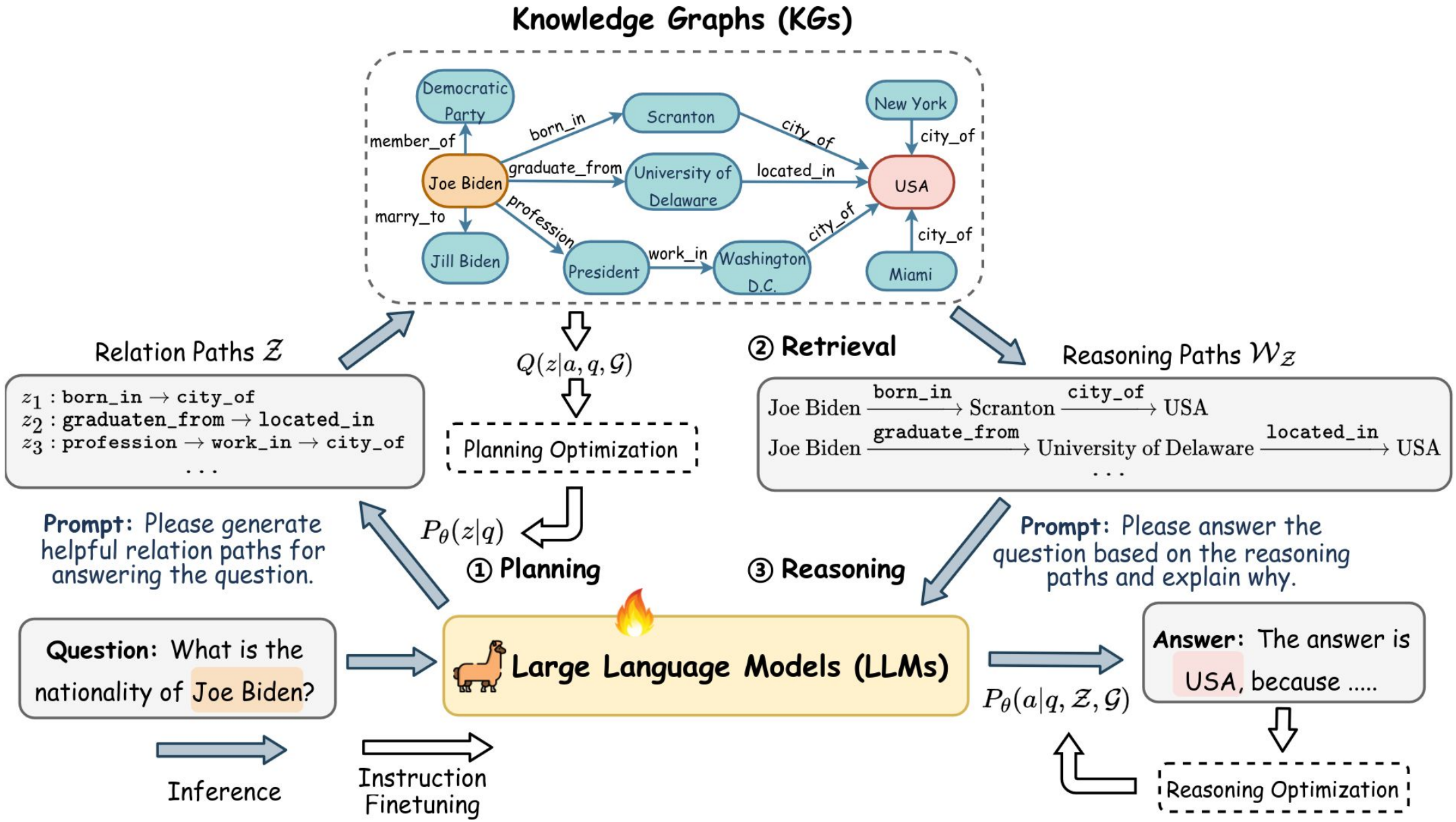
- But there is one more signal to exploit
  - Additional examples of test relation (genre)

Reasoning on Graphs (2024)



- Two ways KG can assist LM-based QA
  - Supplement knowledge
  - Prevent hallucination with path guidance

RoG mechanism



RoG steps

- Planning
  - Select subset of (short) relation type paths, given the question
- Retrieval
  - Expand from anchor nodes, using paths that match the path patterns, thus collecting subgraph
- Reasoning
  - Textify retrieved subgraph, then depend on LLM magic to reason and generate answer

KG-FiD (2021)

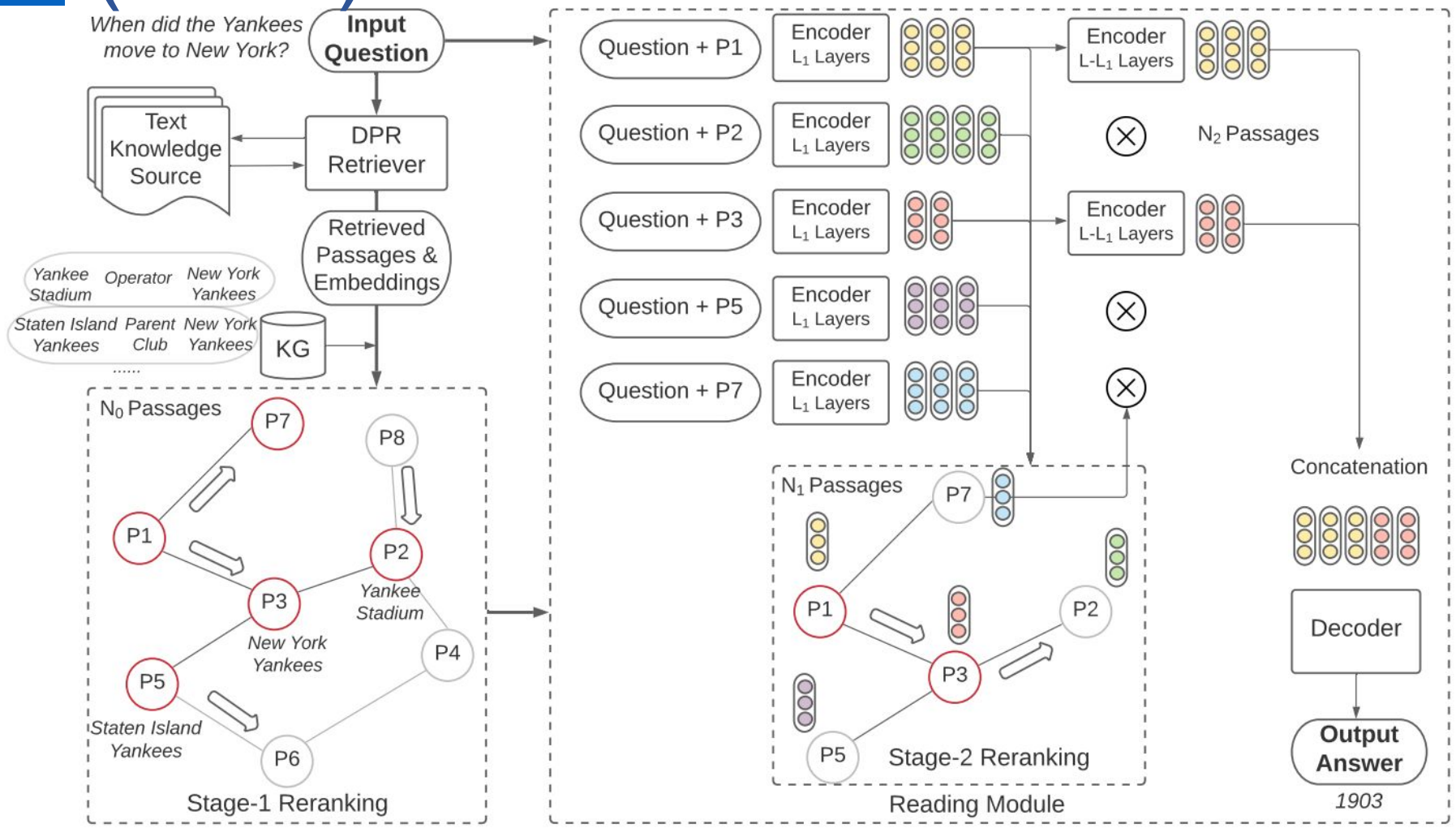
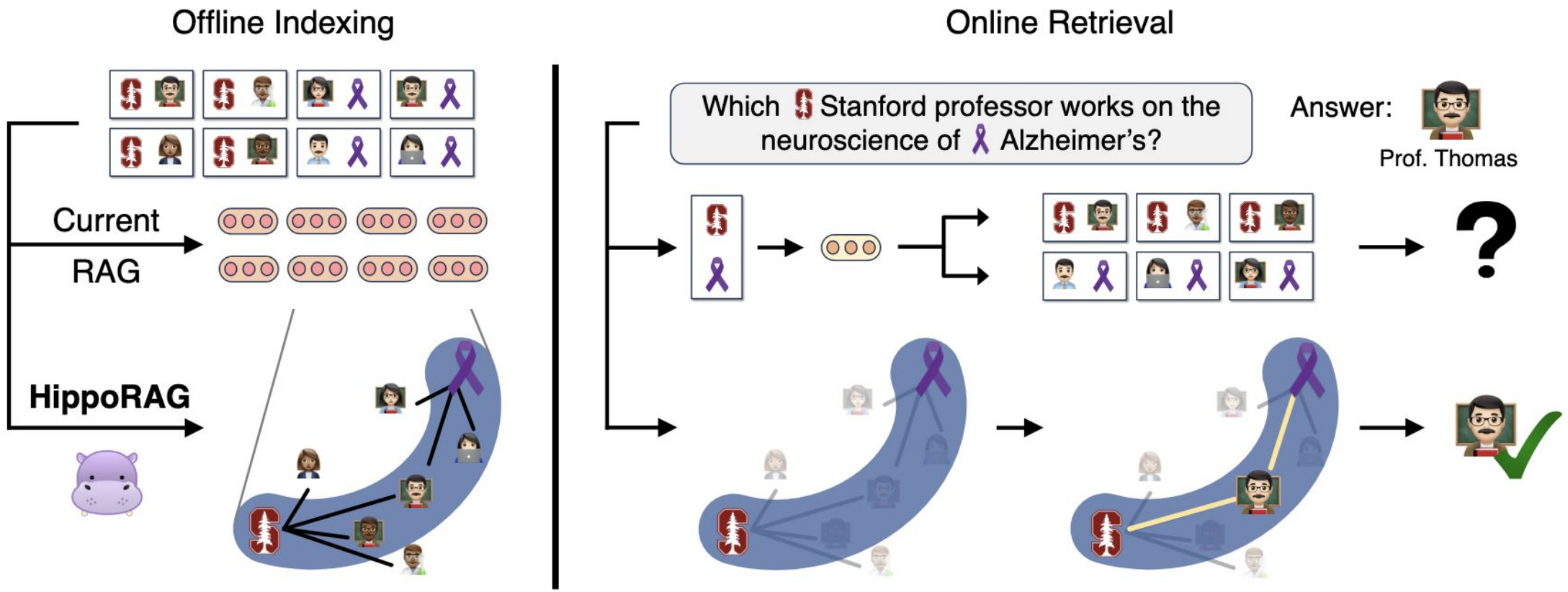


Figure 1: Overall Model Framework. Pi indicates the node of the passage originally ranked the i-th by the DPR retriever, with the article title below it. The left part shows passage retrieval by DPR, passage graph construction based on KG (Section 3.1) and stage-1 reranking (Section 3.2). The right part shows joint stage-2 reranking and answer generation in the reading module (Section 3.3 and 3.4).

KG-FiD details

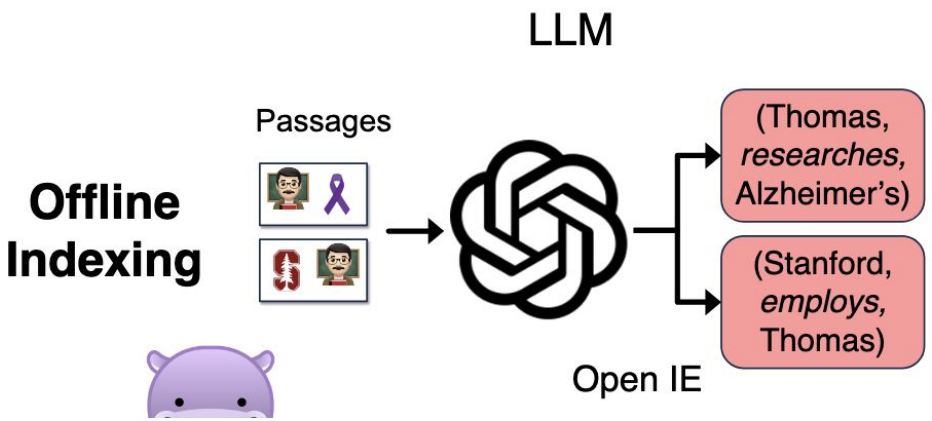
- DPR fetches initial doc subset, each doc z_u is a node
  - DPR uses question embedding $\boldsymbol{q}$
- Build a graph $\mathcal{G}_0$ between these nodes
  - Depends on 1-1 correspondence between DPR corpus and KG
- Stage-1 reranking: Apply graph attention network (GAT) on $\mathcal{G}_0$
  - First-layer node vectors are from DPR (independent of $\boldsymbol{q}$)
  - Last-layer node vectors $\boldsymbol{h}_u$
  - Sort nodes by decreasing $\boldsymbol{q} \cdot \boldsymbol{h}_u$ and retain top nodes $\rightarrow \mathcal{G}_1$
  - GAT trained by retention of gold answer node
- Stage-2 reranking: Now use $\boldsymbol{q}$-contextualized first-layer node embeddings
  - As computed by FiD encoder (smaller number of nodes in $\mathcal{G}_1$)
  - Same GAT-based pruning protocol as in stage-1
- Decode survivors as in FiD using T5

Graph RAG



source: HippoRAG: Neurobiologically Inspired Long-Term Memory for Large Language Models

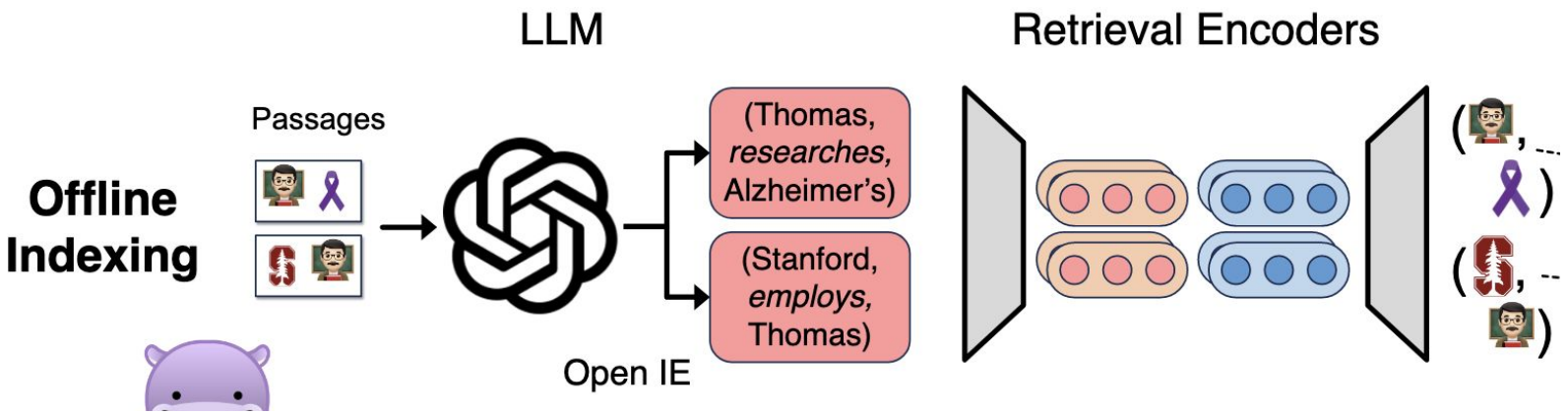
Hippo RAG – Offline Indexing



1. OpenIE - Extract noun phrases & relations

source: HippoRAG: Neurobiologically Inspired Long-Term Memory for Large Language Models

Hippo RAG – Offline Indexing

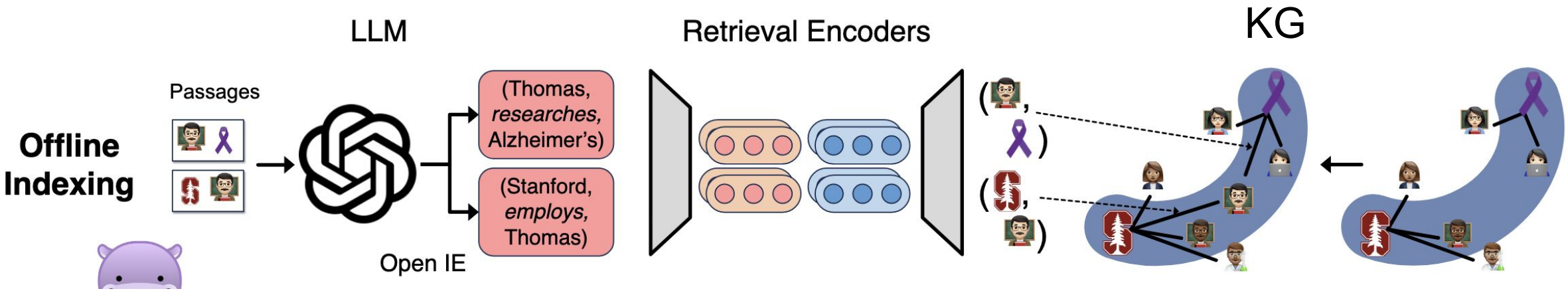


1. OpenIE - Extract noun phrases & relations

2. Add Synonymy relations: Encode noun phrases via encoder & join similar nodes

source: HippoRAG: Neurobiologically Inspired Long-Term Memory for Large Language Models

Hippo RAG – Offline Indexing



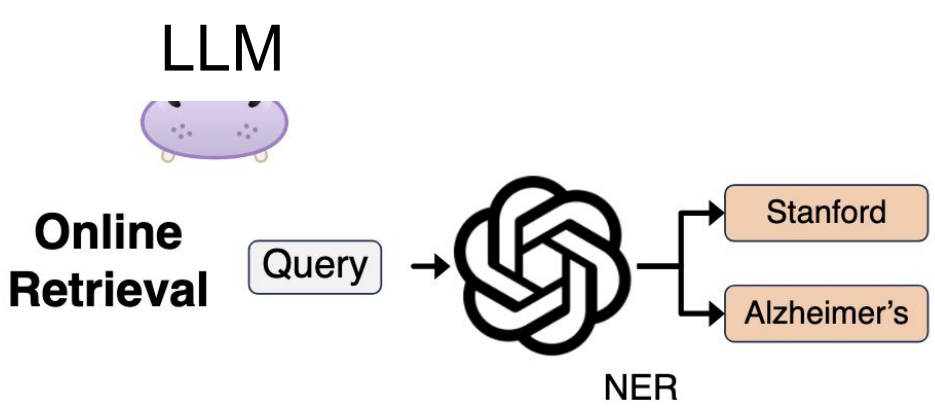
1. OpenIE - Extract noun phrases & relations

2. Add Synonymy relations: Encode noun phrases via encoder & join similar nodes

3. Store node embeddings
4 . Store KG
5. Store noun-phrase Passage id mapping

source: HippoRAG: Neurobiologically Inspired Long-Term Memory for Large Language Models

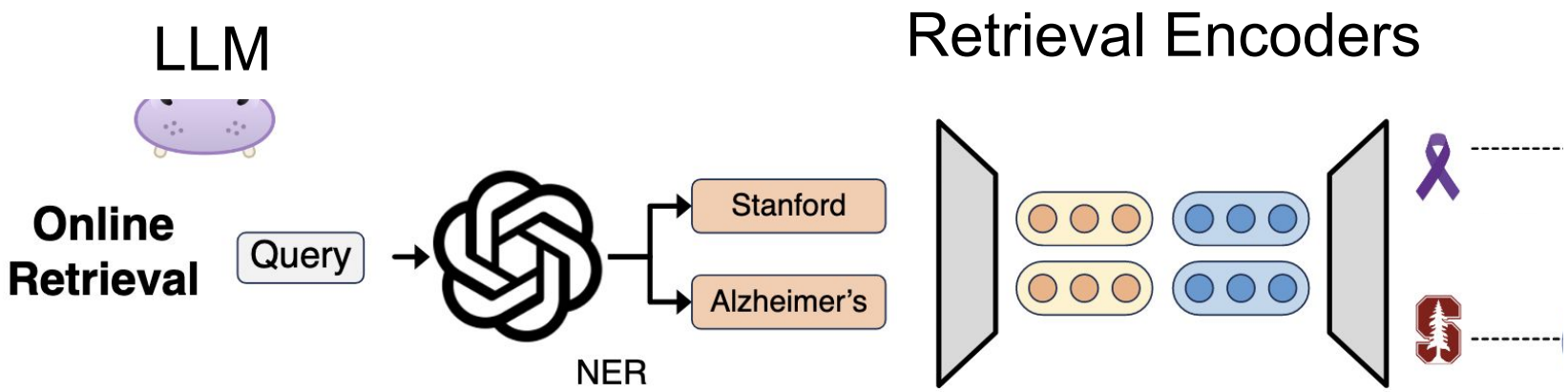
Hippo RAG – Query Time



1. Extract named entities from query

source: HippoRAG: Neurobiologically Inspired Long-Term Memory for Large Language Models

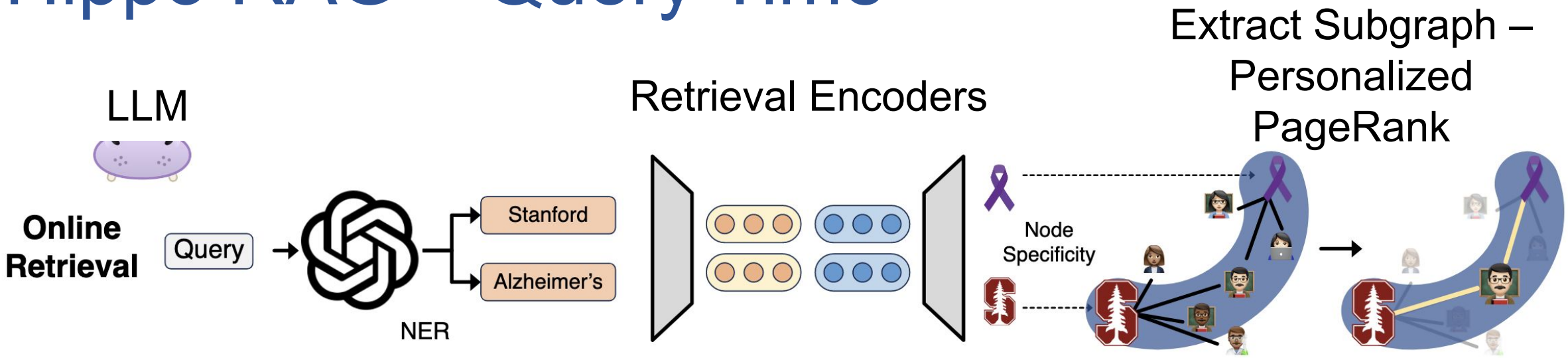
Hippo RAG – Query Time



1. Extract named entities from query

2. Encode named entities
3. Search in the index (recall we indexed node embeddings)

Hippo RAG – Query Time



1. Extract named entities from query

2. Encode named entities
3. Search in the index (recall we indexed node embeddings)

3. Subgraph sampling via PPR – extract nodes (noun phrases) in the nbhrhood

4. Extract the passages that mention the noun phrases

source: HippoRAG: Neurobiologically Inspired Long-Term Memory for Large Language Models

Additional references

- Various paradigms of LMs accessing
  - Passages as items from corpus
  - KGs encoded via GNNs
  - KG subgraphs presented as text

- ACL tutorial focused on passage-RAG
- Online course focused on KG-RAG
- LMs cannot always adequately exploit long retrieval context

Summary and outlook

- Even with long context support, focused retrieval is essential
- Retrieve from passage, graphs, tables, charts, images, speech, video, brain signals, …
- We will focus next on knowledge graphs
- Then tables represented as graphs
- Finally complete a survey of retrieval architectures
  - Integration between LLMs and retrieval systems